4. Python SQLite Game Shop Tutorial – Insert Records

Contents

4. Python SQLite Game Shop Tutorial – Insert Records 1
SQL Tutorial 1
SQLite with Python Tutorials 1
Tutorial 1: Insert Records 1
Do: Insert Data with DB Browser for SQLite 2
Do: sqlite_tutorial_3_insert_records.py 3
Assignment: Add Another Record 6
Assignment Submission 6

Time required: 30 minutes

- Follow all directions carefully and accurately.
- Think of the directions as minimum requirements.

SQL Tutorial

- https://www.w3schools.com/sql/sql_insert.asp

SQLite with Python Tutorials

- SQLite Databases with Python - Full Course – FreeCodeCamp.org
- https://www.sqlitetutorial.net

Tutorial 1: Insert Records

Examples of sample records we will be inserting. PK stands for **P**rimary **K**ey.

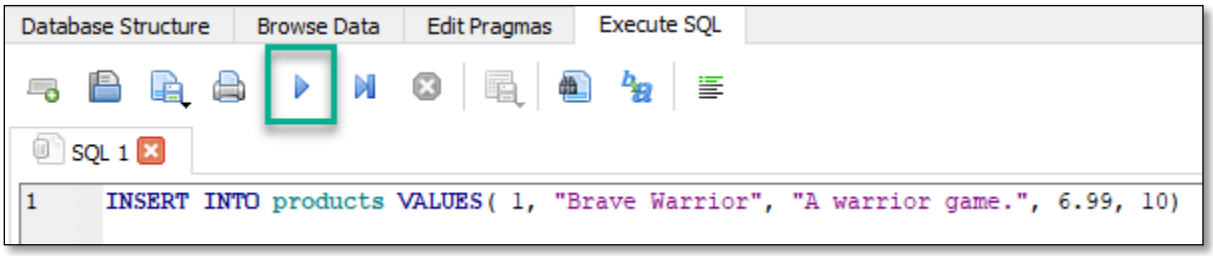
prod_id (PK)	**prod_name**	**prod_desc**	**prod_rank**	**prod_price**	**prod_qty**
1	Minecraft			15.99	1

2	Tetris			12.99	7
3	Pong			2.99	6
4	Guessing Game			.99	200

The product table will track a value for id, name, description, rank, price and quantity. The id is the primary key for the table. A primary key is a special relational database table column (or combination of columns) designated to uniquely identify each table record. You cannot have two id's with the same value

Do: Insert Data with DB Browser for SQLite

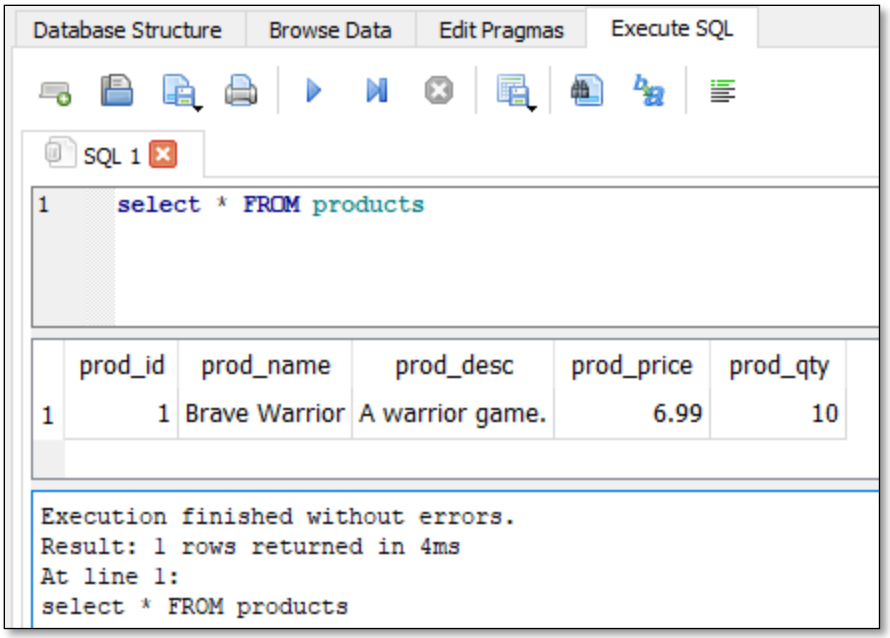
1. Copy **game_shop_2.db** to **game_shop_3.db**
2. In **DB Browser for SQLite** → Open Database **game_shop_3.db**
3. Go to the **Execute SQL** tab.
4. Insert the following SQL code. Add **prod_rank** information.



5. Click the **Execute** button.

```
Execution finished without errors.
Result: query executed successfully. Took 0ms, 1 rows affected
At line 1:
INSERT INTO products VALUES( 1, "Brave Warrior", "A warrior game.", 6.99, 10)
```

6. Show the records as show with the SQL **SELECT** statement.



The SELECT statement is used to select records.

SELECT: Select indicated records

*****: All records

FROM: The table name

Do: sqlite_tutorial_3_insert_records.py

Explanation

- **INSERT INTO products** specifies that we are inserting into the products table.
- **VALUES** says that the values are coming up next.
- The values must match the datatypes of the table fields.

Copy the previous tutorial into a new file: **sqlite_tutorial_4_insert_records.py**

Insert the following changes into the program. You can insert any games you wish. Where it says: **insert your own description**, describe the game you are inserting.

```python
"""
    Name: sqlite_tutorial_3_insert.py
    Author: William Loring
    Created: 01/05/22
    Insert records into the table
"""
# Import sqlite3 database library
import sqlite3


class GameShop():
    def __init__(self, database: str):
        self.database = database

# ------------------------- CREATE TABLE ----------------------------------#
    def create_table(self):
        """Create database and table if not exists."""
        # If everything inside the with sqlite3.connect is successful,
        # connect.commit() and connect.close() are automatically called
        # when the with statement exits
        # Establish a connection to the database file using a
        # with context manager
        with sqlite3.connect(self.database) as connection:

            # Create a cursor object to interact with the database
            cursor = connection.cursor()

            # SQL statement to drop 'products' table if it exists
            # SQL statement to create the 'products' table
            SQL = """
            DROP TABLE IF EXISTS products;
            CREATE TABLE products (
                prod_id       INTEGER PRIMARY KEY,
                prod_name     TEXT,
                prod_desc     TEXT,
                prod_price    REAL,
                prod_qty      INTEGER
            );
            """

            # Execute the SQL script
            cursor.executescript(SQL)

        # Changes are committed automatically after the with handler exits
        # All connections are closed
```

```python
# ------------------------ INSERT RECORDS ---------------------------------#
    def insert_records(self):
        """Insert predefined product records into the 'products' table."""

        # Establish a connection to the database file using the
        # with context manager
        with sqlite3.connect(self.database) as connection:

            # Create a cursor object to interact with the database
            cursor = connection.cursor()

            # SQL statements to insert predefined records
            # into the 'products' table
            SQL = """
            INSERT INTO products VALUES (
                1,
                'Minecraft',
                'Insert description',
                15.99,
                1);
            INSERT INTO products VALUES (
                2,
                'Tetris',
                'Insert description',
                12.99,
                7);
            INSERT INTO products VALUES (
                3,
                'Pong',
                'Insert description',
                .99,
                6);
            INSERT INTO products VALUES (
                4,
                'Guessing Game',
                'Insert description',
                .99,
                200
            );
            """

            # Execute the SQL script
            cursor.executescript(SQL)
```

```python
def main():
    # Create a GameShop instance with the specified database file
    game_shop = GameShop("game_shop_3.db")

    # Create the 'products' table in the database
    game_shop.create_table()
    print("Table created")

    # Insert predefined records into the 'products' table
    game_shop.insert_records()
    print("Records inserted")


# If a standalone program, call the main function
# Else, use as a module
if __name__ == "__main__":
    main()
```

Example run:

```
Table dropped if it exists
Table created
Records inserted
```

Assignment: Add Another Record

1. Add the **prod_rank** field with an integer data type.
2. Add two more game records to the **insert_record()** method.

You will have 6 games in total, each with a product rank.

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.